

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

School of Computer Science and Engineering

SC2104/CE3002: Sensors, Interfacing and Digital Control

Topic 2 - SPI and I^2C Serial Interfaces

Assoc Prof Nicholas Vun

Serial Interfaces

The two most commonly widely used interfaces to connect between microcontroller and digital peripheral ICs (such as sensors, ADC, DAC etc)

i) Serial peripheral Interface (SPI)
ii) Inter-Integrated Circuit (I^2C) bus

Both are serial interfaces

- data is transferred one bit at a time
- reduce the number of digital GPIO pins required in embedded system design

Both are used on the STM32 platform in the Lab exercises

- SPI for OLED display
- I^2C for the IMU sensor

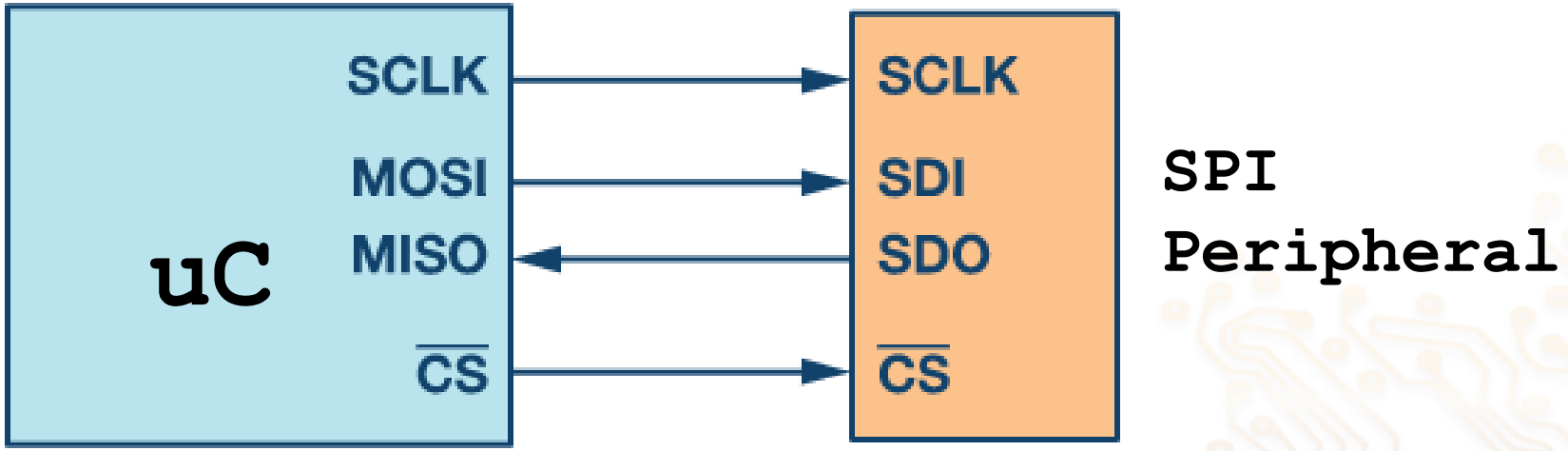
Serial Peripheral Interface

SPI is a serial interface standard originally proposed by Motorola

- for connecting peripherals to its microcontroller using serial interface.

It specifies 3 wires to perform the serial bit data transfer

SDI : serial data input (for receiving data)
SDO : serial data output (for transmitting data)
SCLK : serial clock (reference clock)

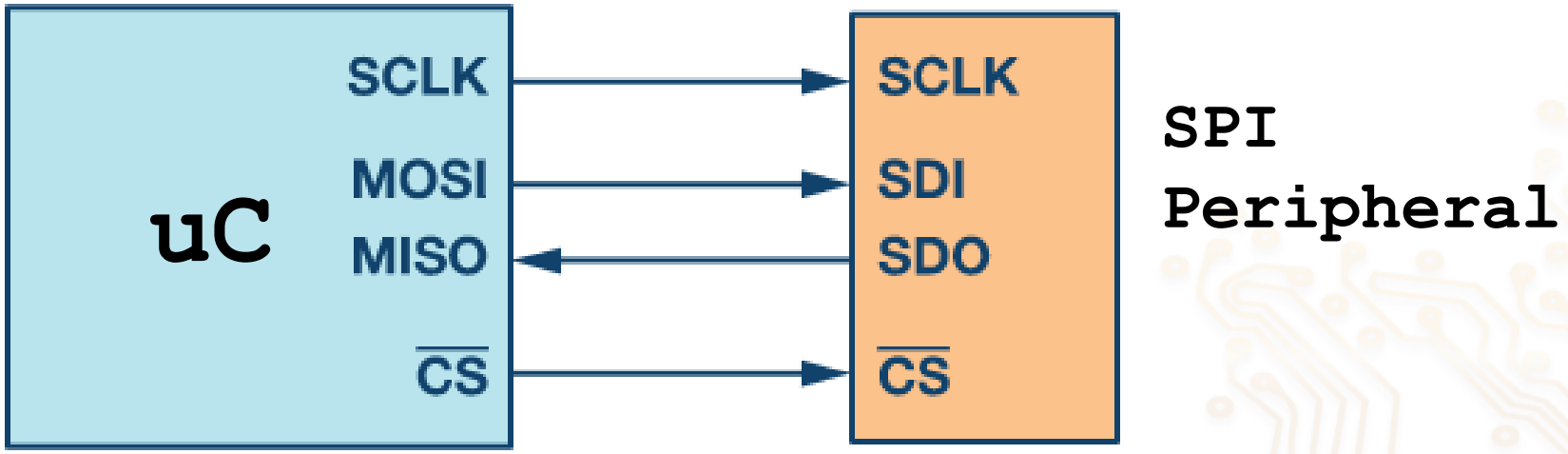


Serial Peripheral Interface Pins

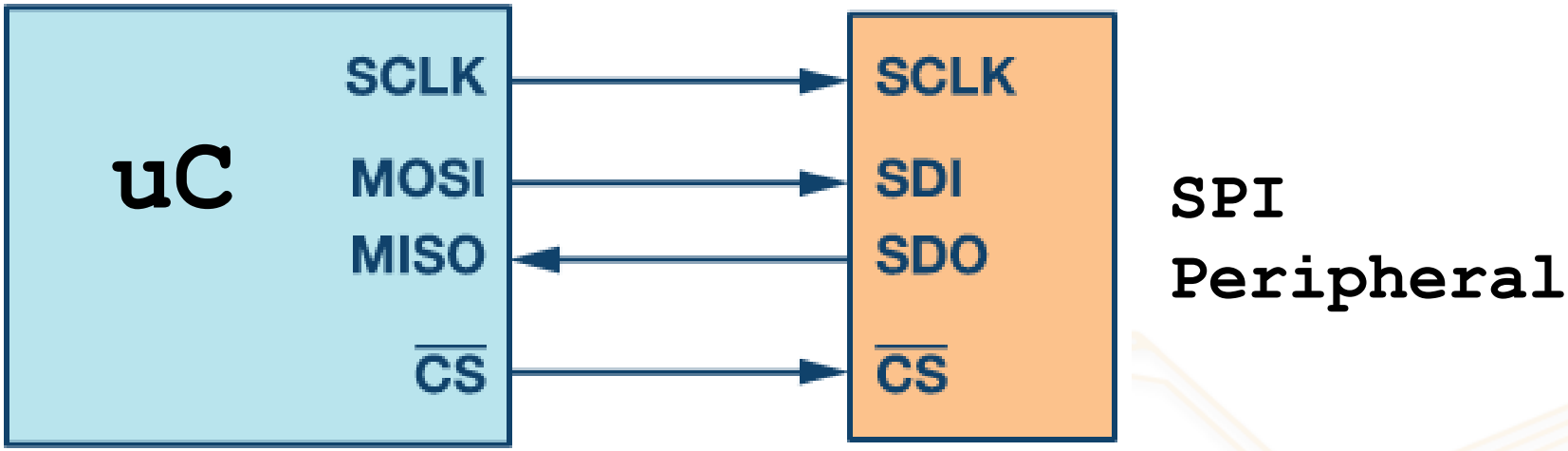
In addition to the three specified SPI bus signals

- peripheral with SPI interface also typically provides an extra control pin (e.g. Chip-Select CS#)
- allows it to be selected by the processor for data transfer

Hence SPI peripheral usually can only perform data transfer under the control of the processor.



Serial Peripheral Interface SCLK



Clock signal SCLK is generated by the processor.

- data bits are sampled based on the edges of the SCK signal
- frequency can run up to a few MHz range
  - hence enable transfer bit rate in the range of few Mbps

With SDI and SDO on separate wires

- SPI can support full duplex synchronous data transfer between the processor and the peripheral(s)

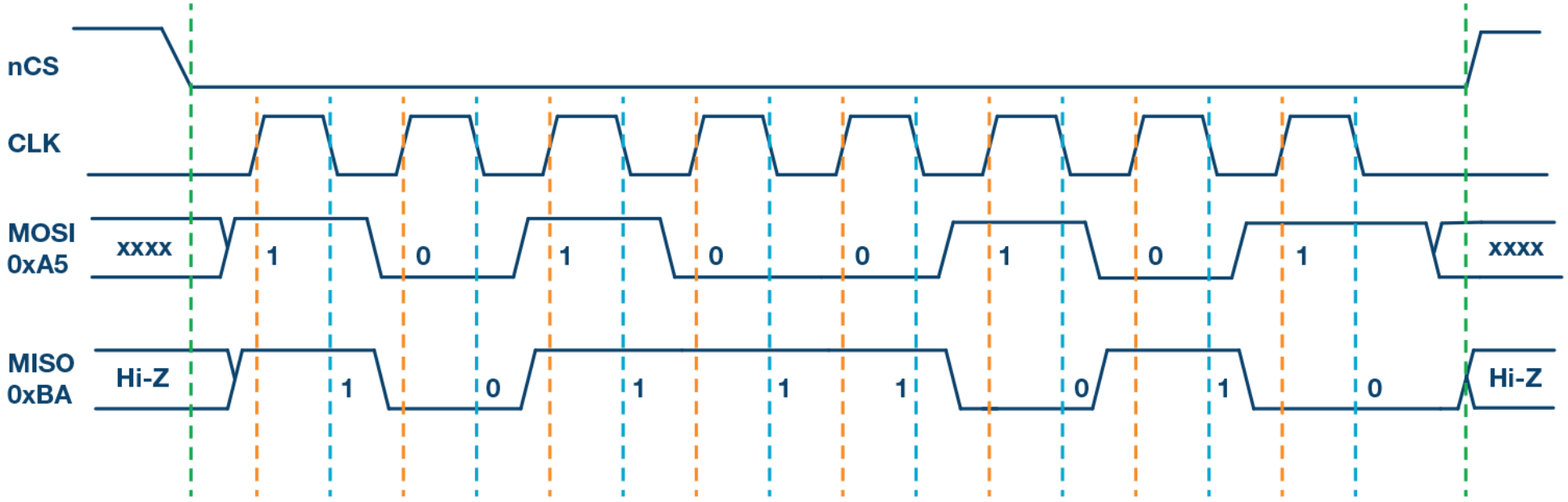
Serial Peripheral Interface Data Sampling

Clock signal SCLK is used as reference for the data sampling

Depending on the specific design

- data bits can be sampled on the rising edge of the SCLK signal (as shown in the following diagram – Yellow lines)
- while data are shifted on SCLK falling edge (Blue lines)

or vice versa



SPI Data Protocol

Data protocol is used to specify the format used in the data transfer

- such that both parties can understand each other.

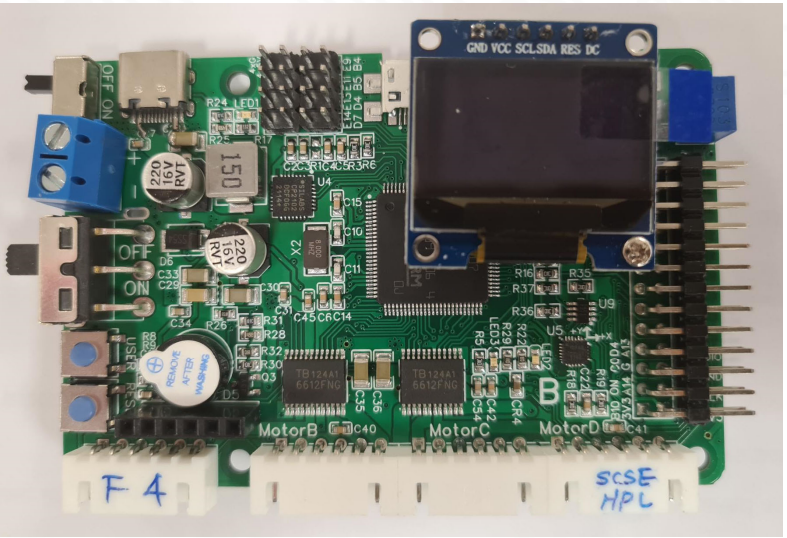
But SPI doesn't specify any data protocol to be used on its bus

- it only provides the signaling and connection requirements

Hence number of bits, and meaning of each bit varies from peripheral to peripheral

- based on the implementation of each peripheral

Case Study – OLED Display on the STM32 Platform





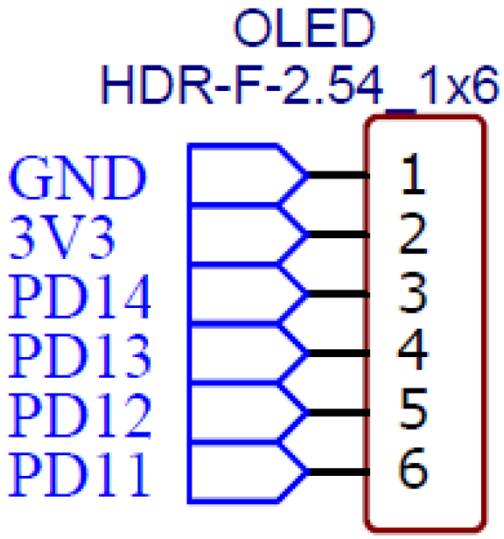
Interface pins:

- SCL* ≡ SCLK
- SDA* ≡ SDI
- RES = Reset the module
- DC = Data/Commands#
- SDO = No SDO since no data to be sent by the display
- CS# = hardwired to 0V

(*I^2C pin labels - see later)

Case Study – OLED Display on the STM32 Platform





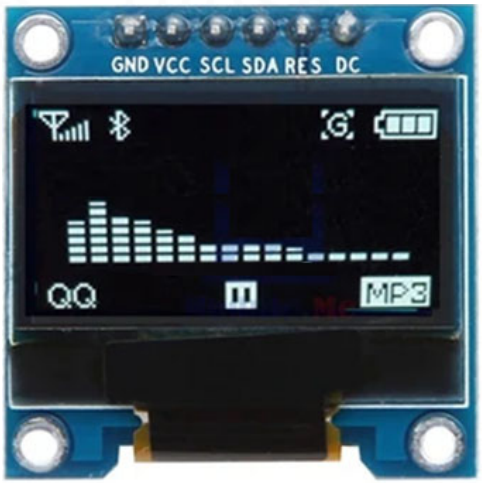
The SCL, SDA, RES and DC pins are to be be driven through GPIOs of the STM32 microcontroller using the SPI protocol

- PD11 to PD14

However, these STM32 pins do not support SPI interface

- hence need to use software emulated 'bit banging'

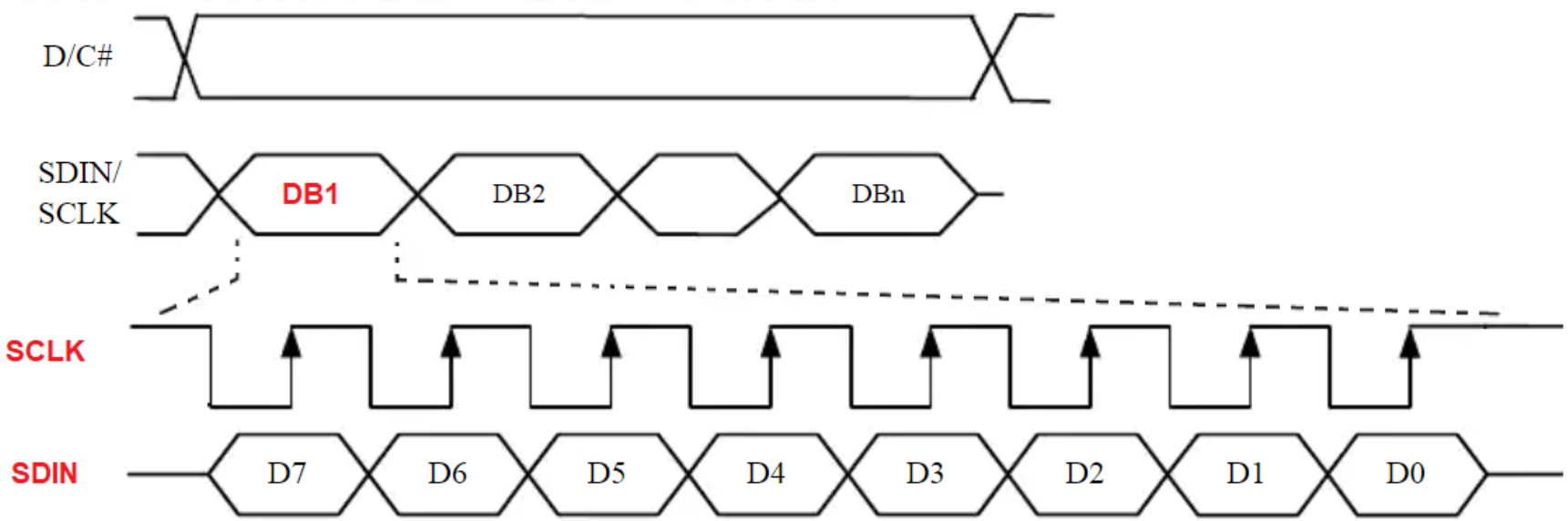
Case Study – OLED Display on the STM32 Platform



The display controller for the OLED display is SSD1306

- RES pin – Reset control pin.
  When the pin is LOW, initialization of the chip is executed.
  This pin needs to be HIGH during normal operation.
- DC pin - Data/Command# control pin.
  When it is HIGH, the data is treated as data for display.
  When it is pulled LOW, the data will be treated as command and transferred to the command register.

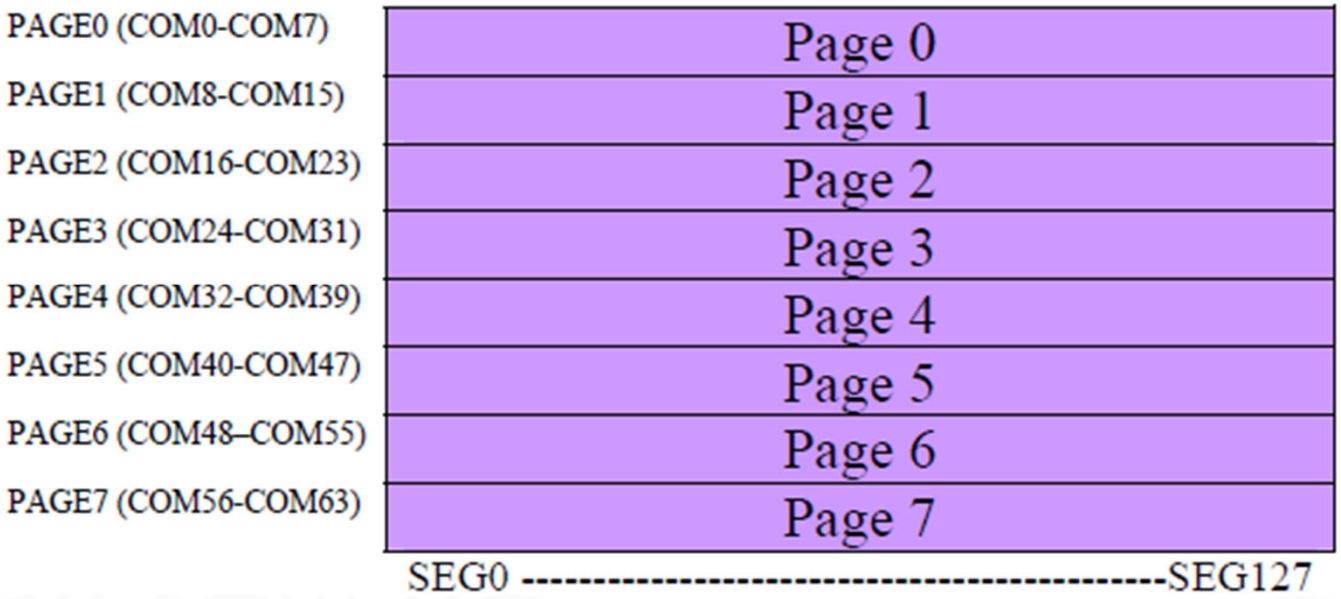
Case Study – Timing diagram



D/C# signal

- used to indicate how the signal on the SDIN (i.e. SDI) pin is to be interpreted.
- LOW – SDIN data is interpreted as a command
  - data will be decoded and written to the corresponding command register.
- HIGH - SDIN data is interpreted as display data written to Graphic Display Data RAM (GDDRAM).

Case Study – GDDRAM bit map



The GDDRAM is a bit mapped static RAM

- holding the bit pattern to be displayed.
- size of the RAM is 128 x 64 bits
- RAM is divided into eight pages
  - from PAGE0 to PAGE7

(Library functions will be provided to be used in the lab to display texts and numbers on the display)

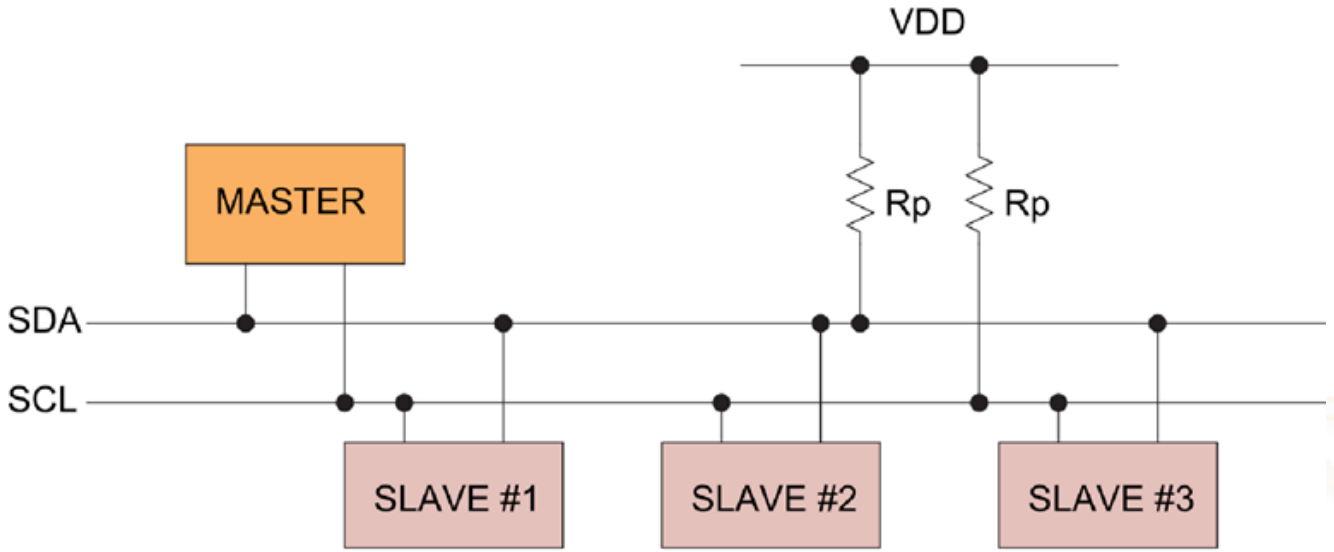
Inter-Integrated Circuit (I^2C)

The Inter-Integrated Circuit (I^2C) bus is a two wire serial interface

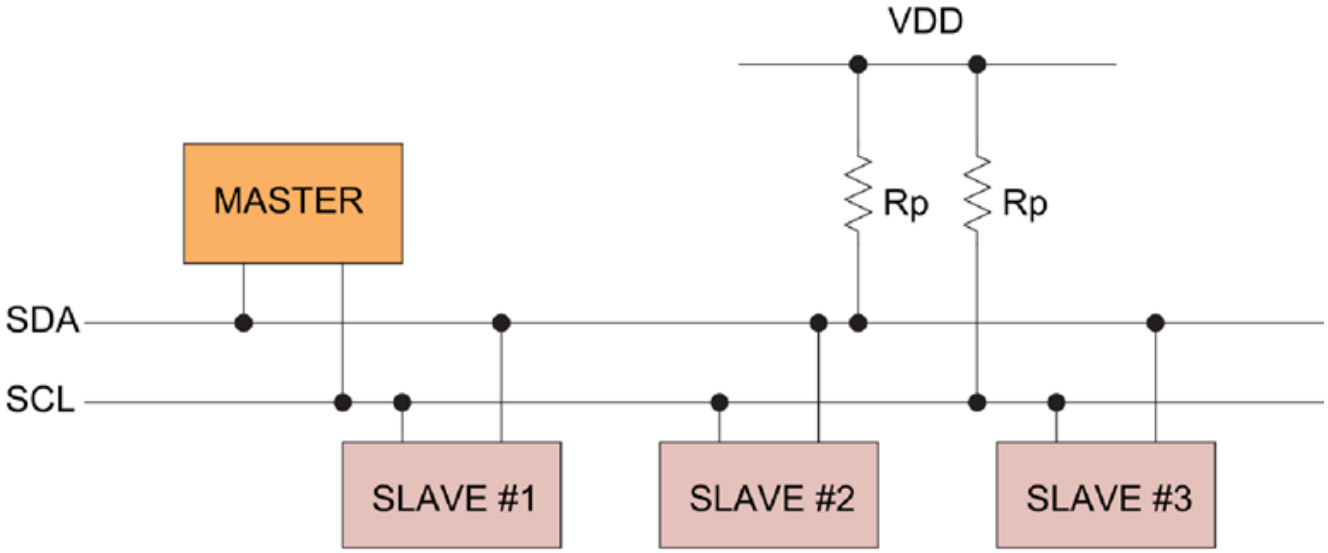
- originally developed by the Phillips Corporation for use in consumer products

Connections are minimized by using

- a Serial Data line (SDA)
- a Serial Clock line (SCL)



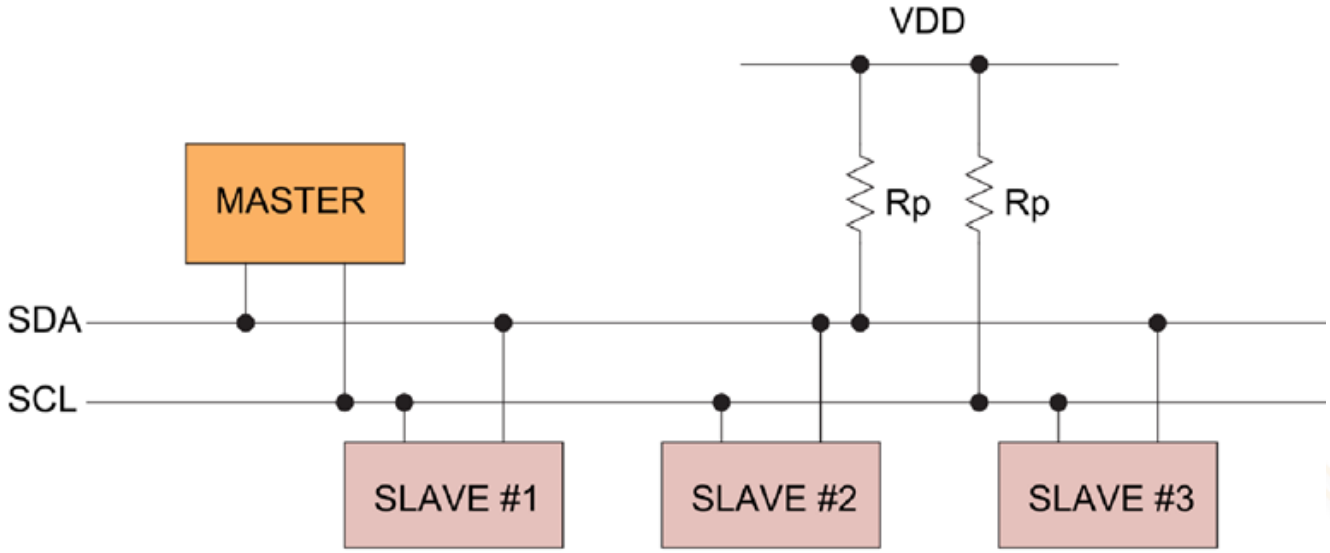
I²C Master/Slave Setup



I²C communication protocol follows a master/slave hierarchy

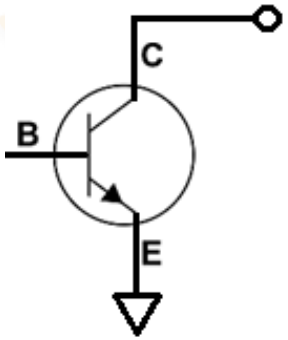
- Master (μP/μC) is the device that clocks the bus
  - addresses the slaves and writes/reads data to/from registers in the slaves.
- slaves are devices (e.g., IMU) that respond only when addressed by the master
  - through their unique address.

I²C Master/Slave Setup



I²C compatible devices connect to the bus with

- open collector pins.



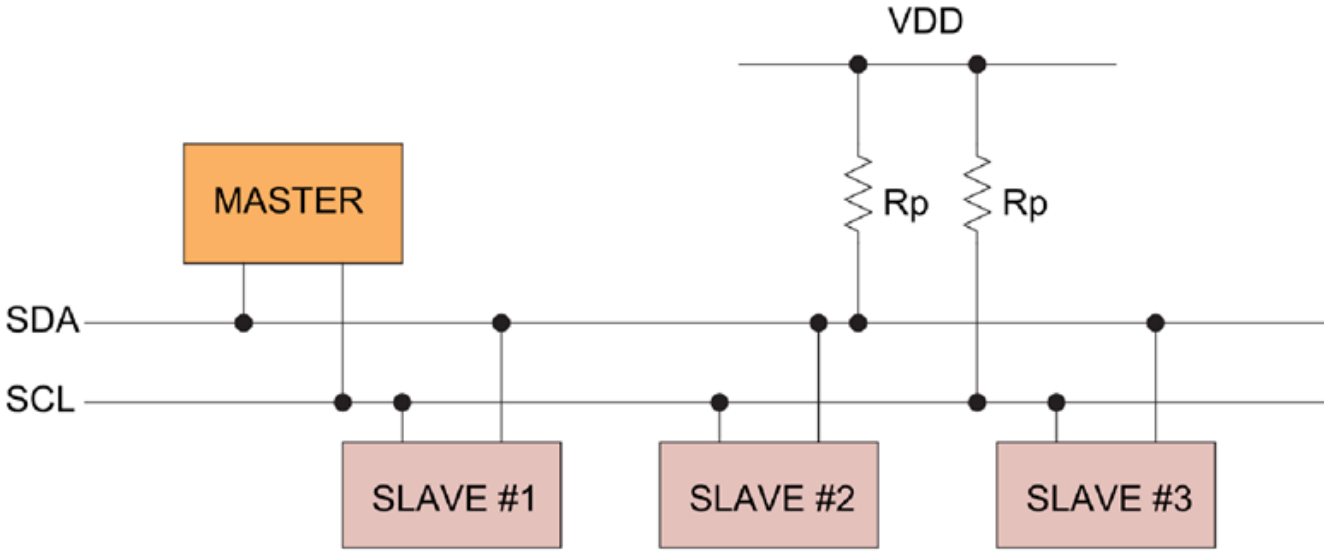
When there is no transmission of data

- the I²C bus lines stay idle in a HIGH state
  - the lines are passively pulled high by Rp resistors.

Transmission occurs by toggling the lines

- by pulling it LOW and releasing it to HIGH

I²C Master/Slave Operation



Bits are synchronized with the SCL clock

- Standard data transfer rate is 100kbits/s
- Fast Mode transfer rate is 400kbits/s.

The I²C bus can support multiple devices

- limitation is the capacitance on the bus (400pF)
- and the address space (128 unique addresses)

Case Study: MPU-6050 (on the STM32 Board)

A 6-Axis IMU based Motion Sensor

- 3-axis gyroscope
- 3-axis accelerometer

I²C Interface

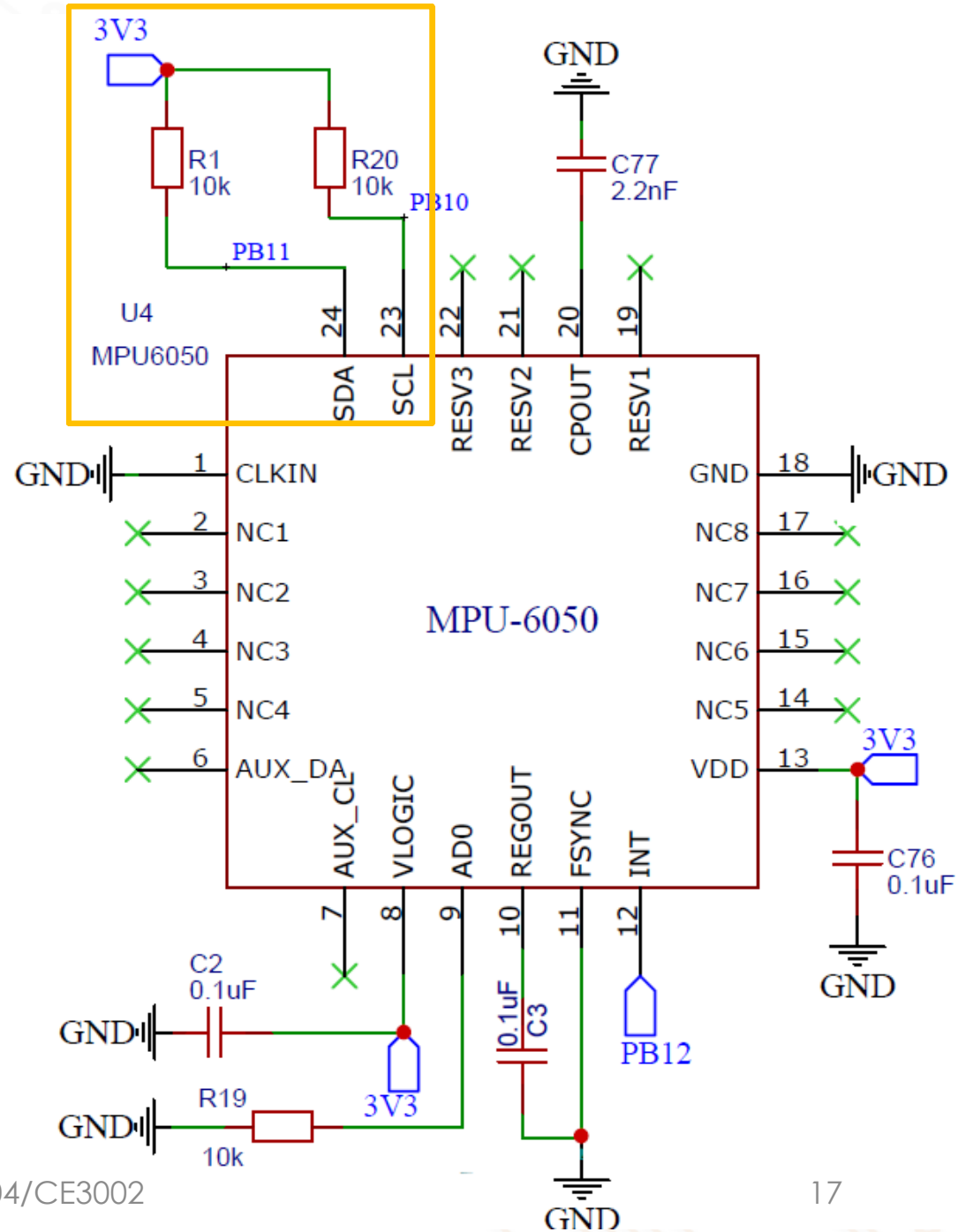
- SCL connects to PB10
- SDA connects to PB11

Interrupt pin – PB12

Address:

- 7 bits
- Default value = 110100x
  - x depends on AD0 pin

Q: What is its address?



Data Transmission Protocol

I^2C data packets are sent in 8-bit bytes comprising of

- slave address
- register number
- data to be transferred
  - no restriction to the number of bytes transmitted per data transfer

Transmission is either a read or write operation, build upon a series of sub-protocols:

- start and stop conditions
- repeated start bits
- address byte
- data transfer bits
- acknowledge/not acknowledge bits.

Address Byte

A slave address is sent in 8-bit byte format

- msb first
- but with the last (lowest) bit signifies whether the transaction is to read or write command to the slave.

Effectively, only the upper 7 bits constitute the slave address

- address space of 128 unique addresses for addressing up to 128 slaves.

Example: Address of the IMU on the STM32C platform

- 1101000b
- (0x68<<1)

Operations

If the first seven bits match a SLAVE device's address

- it continues to read the 8th bit (READ/WRITE# bit) of the address byte.
- LOW in the 8th bit => MASTER will write data
- HIGH in the 8th bit => MASTER will read data from the SLAVE device.

Once the address byte has been read by the SLAVE

- SLAVE acknowledges the transfer by sending an acknowledge bit
- SLAVE then assume the role of a slave-transmitter or slave-receiver
    - depending on the READ/WRITE# bit sent.

Command Byte

When writing to or reading from a specific register in a SLAVE

- master must first point to the specific register
  - by writing the register address after the SLAVE has been addressed.

The register address can be considered as a special data byte

- to avoid confusion - it is often classified as a Command byte.

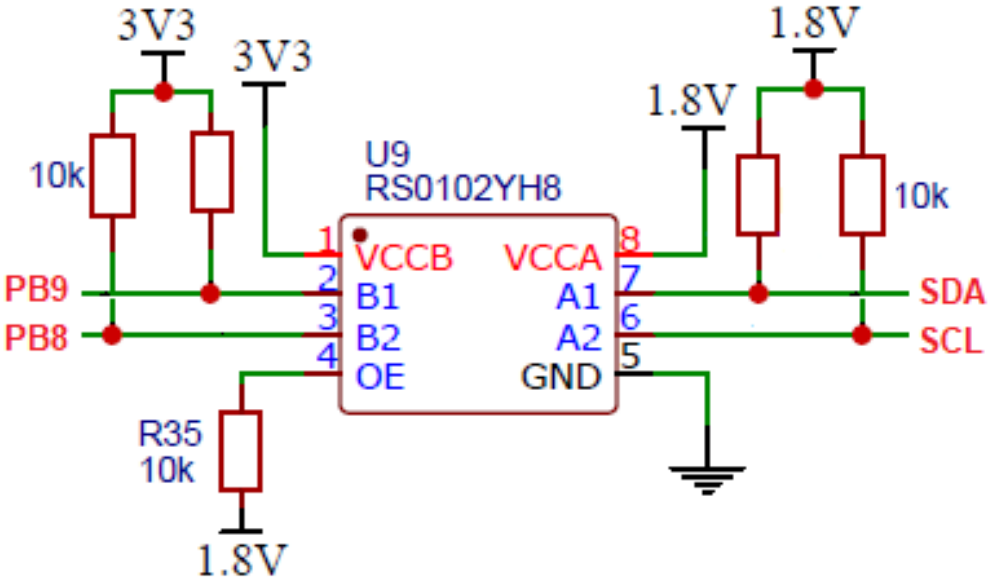
Slave address Register Number Data

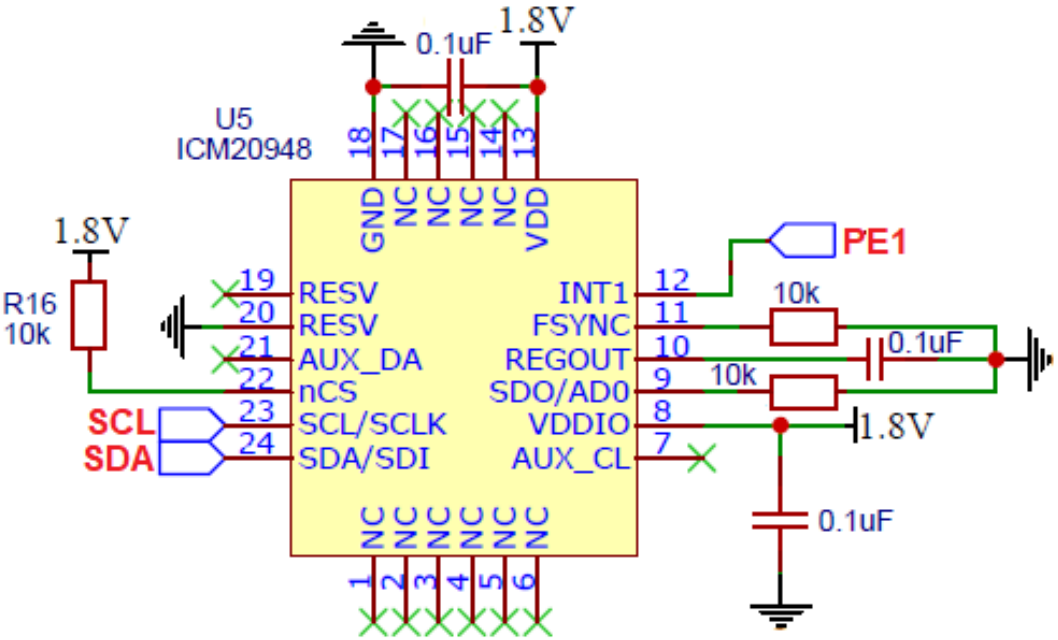
S	ADDRESS	$\overline{W}$	A	COMMAND	A	DATA	A	DATA	A	P
	1 1 0 a3:a0	0	0	X X b5:b0	0	b7:b0	0	b7:b0	0	

Case Study – ICM-20946 (used in MDP)

A 9-Axis IMU based Motion Sensor

- 3-axis gyroscope, 3-axis accelerometer, 3-axis compass
- VCC = 1.8V





7-bit address =1101000b (With ADO=0)

Case Study – ICM-20946

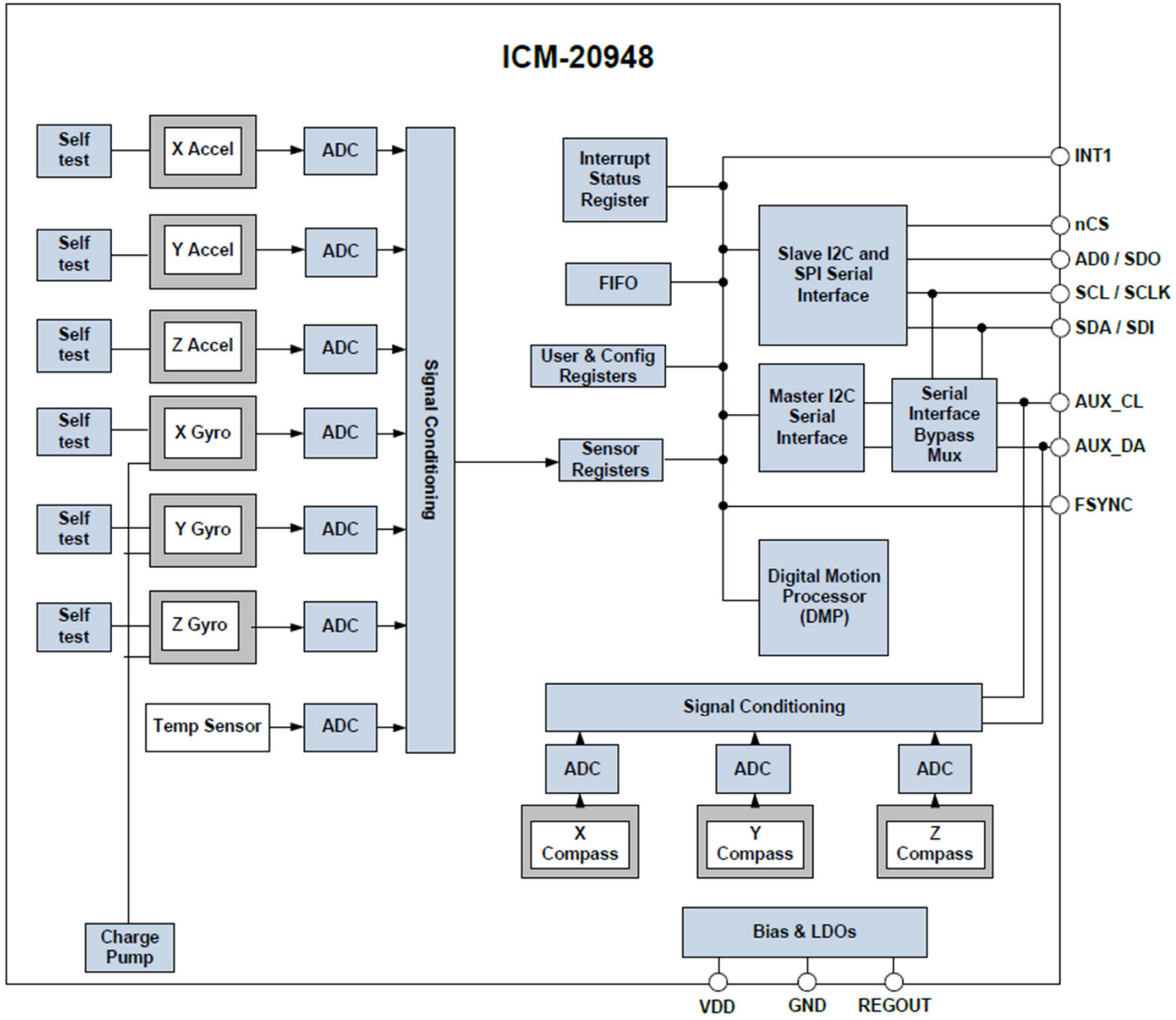


Figure 5. ICM-20948 Block Diagram

Data Registers in Sensor

Sensor contains data registers holding the latest gyro, accelerometer, auxiliary sensor, and temperature measurement data etc.

7.1 USER BANK 0 REGISTER MAP

ADDR (HEX)	ADDR (DEC.)	REGISTER NAME	SERIAL I/F	Data
00	0	WHO_AM_I	R	WHO_AM_I[7:0]
2D	45	ACCEL_XOUT_H	R	ACCEL_XOUT_H[7:0]
2E	46	ACCEL_XOUT_L	R	ACCEL_XOUT_L[7:0]
2F	47	ACCEL_YOUT_H	R	ACCEL_YOUT_H[7:0]
30	48	ACCEL_YOUT_L	R	ACCEL_YOUT_L[7:0]
31	49	ACCEL_ZOUT_H	R	ACCEL_ZOUT_H[7:0]
32	50	ACCEL_ZOUT_L	R	ACCEL_ZOUT_L[7:0]
33	51	GYRO_XOUT_H	R	GYRO_XOUT_H[7:0]
34	52	GYRO_XOUT_L	R	GYRO_XOUT_L[7:0]
35	53	GYRO_YOUT_H	R	GYRO_YOUT_H[7:0]
36	54	GYRO_YOUT_L	R	GYRO_YOUT_L[7:0]
37	55	GYRO_ZOUT_H	R	GYRO_ZOUT_H[7:0]
38	56	GYRO_ZOUT_L	R	GYRO_ZOUT_L[7:0]

ID = 0xEA

Summary

SPI and I^2C are the most common serial interface used for many modern Sensors and peripherals.

Both are synchronous driven using a reference a clock signal.

SPI selects the device using the CS#

- use separate wires for transmitting data bit
  - no ‘turn-around’ time need
  - much faster than I^2C

I^2C selects the device using address

- shared wire for transmitting data bit
  - need ‘turn-around’ time